

LAB DAY 7

NAME: AGILAN VIYASH

REG. NO.: 192511346

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

COIN CHANGE – SUPERMARKET BILLING

PYTHON CODE:

```
# Coin Change - Supermarket Billing
```

```
coins = [1, 2, 5, 10]
```

```
amount = 28
```

```
coins.sort(reverse=True)
```

```
result = []
```

```
remaining = amount
```

```
for coin in coins:
```

```
    while remaining >= coin:
```

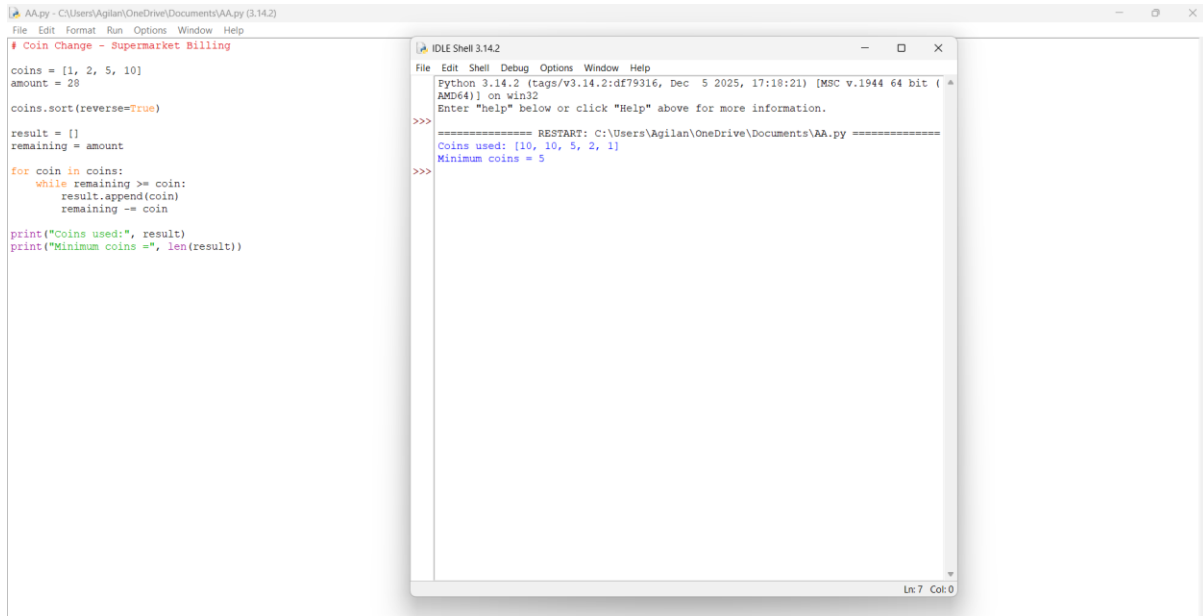
```
        result.append(coin)
```

```
        remaining -= coin
```

```
print("Coins used:", result)
```

```
print("Minimum coins =", len(result))
```

EXECUTION & OUTPUT:



COIN CHANGE – PARKING METER

PYTHON CODE:

Coin Change - Parking Meter

coins = [1, 5, 10, 25]

amount = 63

coins.sort(reverse=True)

result = []

remaining = amount

for coin in coins:

 while remaining >= coin:

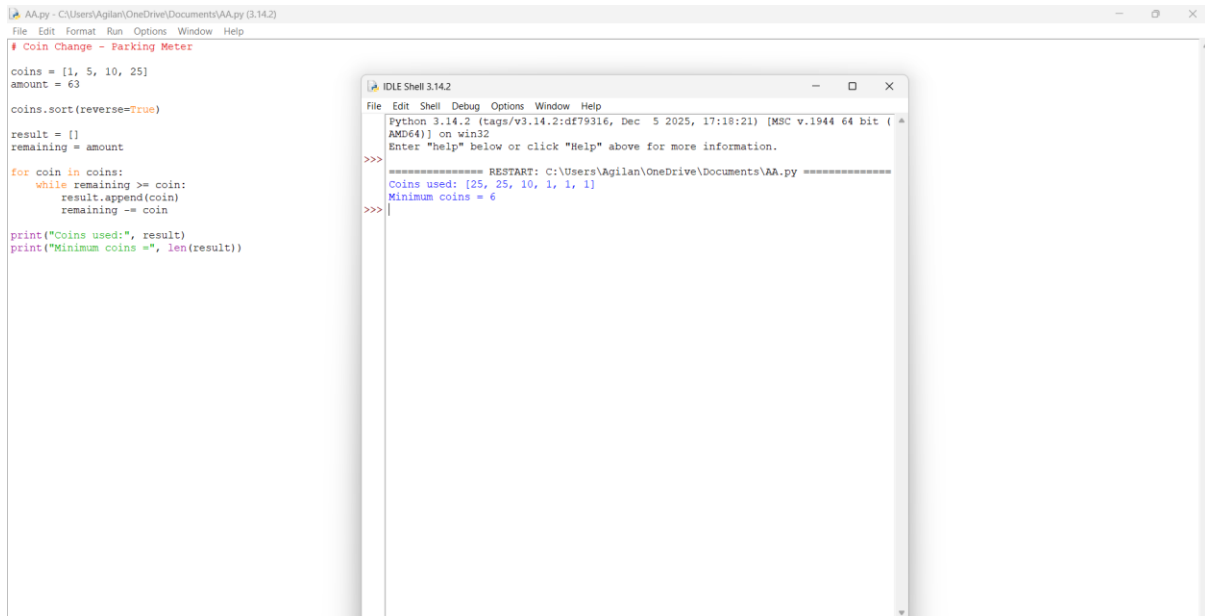
 result.append(coin)

 remaining -= coin

print("Coins used:", result)

```
print("Minimum coins =", len(result))
```

EXECUTION & OUTPUT:



COIN CHANGE – TICKET COUNTER

PYTHON CODE:

```
# Coin Change - Ticket Counter
```

```
coins = [1, 3, 7, 10]
```

```
amount = 24
```

```
coins.sort(reverse=True)
```

```
result = []
```

```
remaining = amount
```

```
for coin in coins:
```

```
    while remaining >= coin:
```

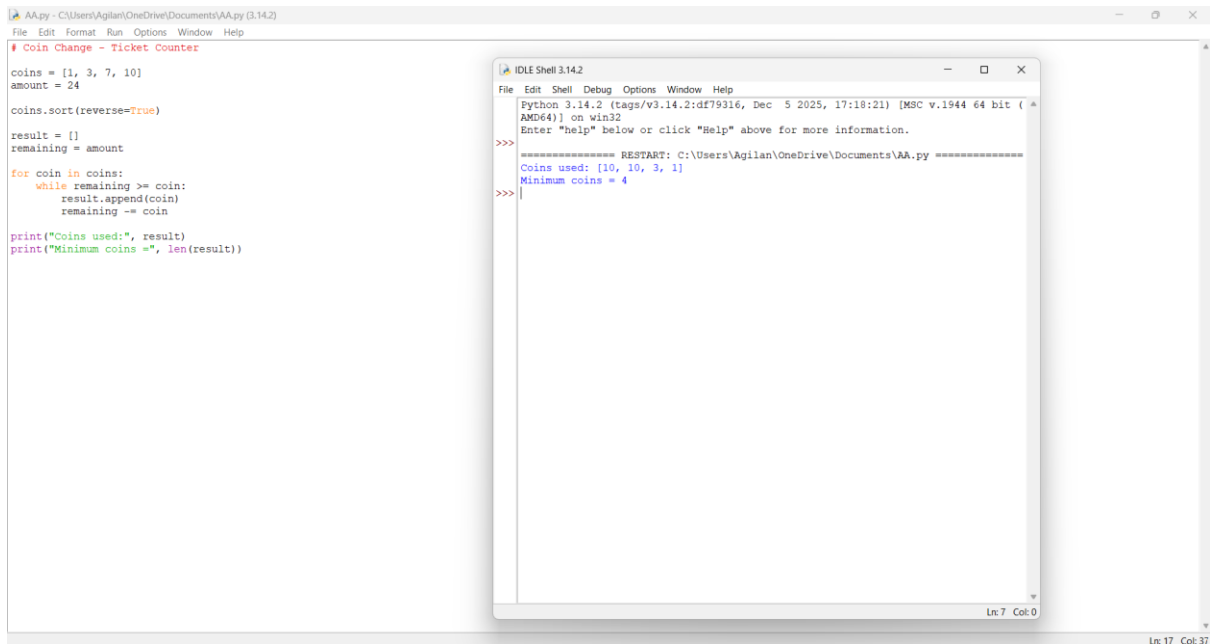
```
        result.append(coin)
```

```
remaining -= coin
```

```
print("Coins used:", result)
```

```
print("Minimum coins =", len(result))
```

EXECUTION & OUTPUT:



FRACTIONAL KNAPSACK – FOOD DELIVERY TRUCK

PYTHON CODE:

```
# Fractional Knapsack - Food Delivery Truck
```

```
weights = [10, 20, 30]
```

```
profits = [60, 100, 120]
```

```
capacity = 50
```

```
items = []
```

```
for i in range(len(weights)):
```

```
    ratio = profits[i] / weights[i]
```

```

        items.append((ratio, weights[i], profits[i]))

# Sort according to profit/weight ratio
items.sort(reverse=True)

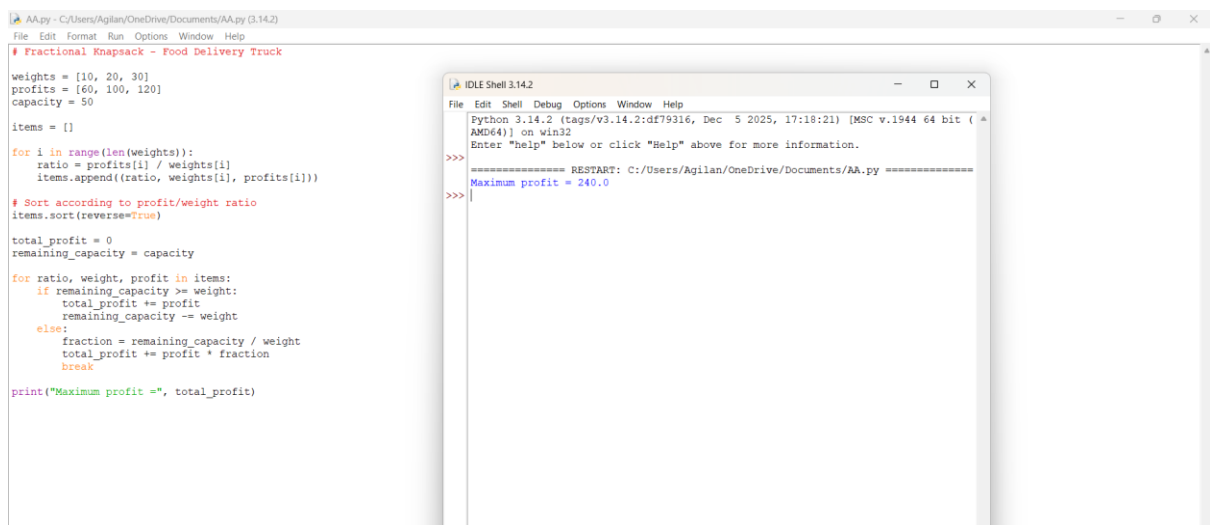
total_profit = 0
remaining_capacity = capacity

for ratio, weight, profit in items:
    if remaining_capacity >= weight:
        total_profit += profit
        remaining_capacity -= weight
    else:
        fraction = remaining_capacity / weight
        total_profit += profit * fraction
        break

print("Maximum profit =", total_profit)

```

EXECUTION & OUTPUT:



The screenshot displays a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The code in the editor is a Fractional Knapsack implementation for a 'Food Delivery Truck'. It defines weights [10, 20, 30] and profits [60, 100, 120] with a capacity of 50. The code calculates the maximum profit by sorting items by their profit-to-weight ratio and selecting items until the capacity is reached. The output of the program is shown in the IDLE Shell window, which displays 'Maximum profit = 240.0'.

```

# Fractional Knapsack - Food Delivery Truck

weights = [10, 20, 30]
profits = [60, 100, 120]
capacity = 50

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

# Sort according to profit/weight ratio
items.sort(reverse=True)

total_profit = 0
remaining_capacity = capacity

for ratio, weight, profit in items:
    if remaining_capacity >= weight:
        total_profit += profit
        remaining_capacity -= weight
    else:
        fraction = remaining_capacity / weight
        total_profit += profit * fraction
        break

print("Maximum profit =", total_profit)

```

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Maximum profit = 240.0
>>>

```

FRACTIONAL KNAPSACK – GOLD TRANSPORT

PYTHON CODE:

```
# Fractional Knapsack - Gold Transport
```

```
weights = [5, 10, 15]
```

```
profits = [30, 40, 45]
```

```
capacity = 20
```

```
items = []
```

```
for i in range(len(weights)):
```

```
    ratio = profits[i] / weights[i]
```

```
    items.append((ratio, weights[i], profits[i]))
```

```
items.sort(reverse=True)
```

```
total_profit = 0
```

```
remaining_capacity = capacity
```

```
for ratio, weight, profit in items:
```

```
    if remaining_capacity >= weight:
```

```
        total_profit += profit
```

```
        remaining_capacity -= weight
```

```
    else:
```

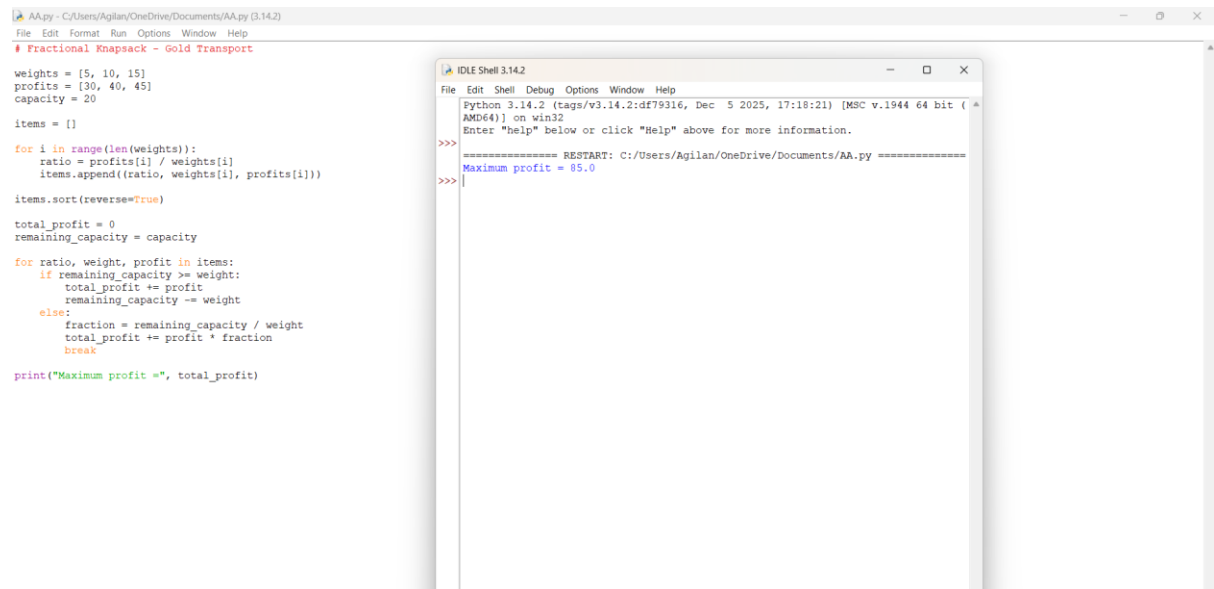
```
        fraction = remaining_capacity / weight
```

```
        total_profit += profit * fraction
```

```
        break
```

```
print("Maximum profit =", total_profit)
```

EXECUTION & OUTPUT:



FRACTIONAL KNAPSACK – RELIEF MATERIAL LOADING

PYTHON CODE:

```
# Fractional Knapsack - Relief Material Loading
```

```
weights = [4, 8, 2, 6]
```

```
profits = [20, 40, 14, 30]
```

```
capacity = 10
```

```
items = []
```

```
for i in range(len(weights)):
```

```
    ratio = profits[i] / weights[i]
```

```
    items.append((ratio, weights[i], profits[i]))
```

```
items.sort(reverse=True)
```

```
total_profit = 0
```

```
remaining_capacity = capacity
```

```
for ratio, weight, profit in items:
```

```
    if remaining_capacity >= weight:
```

```
        total_profit += profit
```

```
        remaining_capacity -= weight
```

```
    else:
```

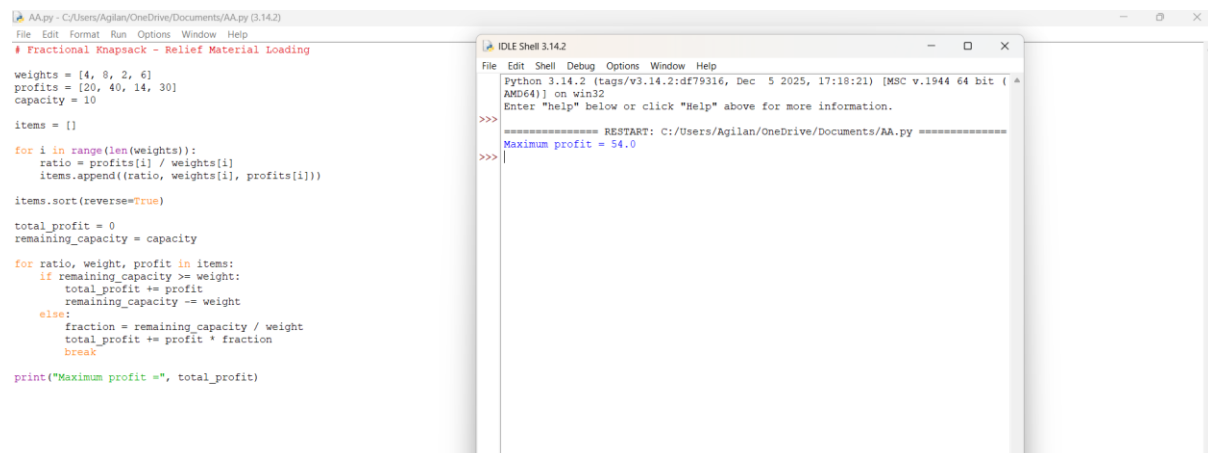
```
        fraction = remaining_capacity / weight
```

```
        total_profit += profit * fraction
```

```
        break
```

```
print("Maximum profit =", total_profit)
```

EXECUTION & OUTPUT:

The image shows a screenshot of a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The code in the editor is a Fractional Knapsack algorithm. It defines weights [4, 8, 2, 6], profits [20, 40, 14, 30], and capacity 10. It sorts items by profit-to-weight ratio in descending order. The algorithm iterates through items, adding them to the knapsack if they fit, or taking a fraction of them if they don't. The final output is 'Maximum profit = 54.0'. To the right, the IDLE Shell window shows the execution output: 'Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32' followed by 'Enter "help" below or click "Help" above for more information.' and the output 'Maximum profit = 54.0'.

JOB SEQUENCING – SOFTWARE COMPANY TASKS

PYTHON CODE:

```
# Job Sequencing - Software Company Tasks
```

```
jobs = ['A', 'B', 'C', 'D']
```



```

deadlines = [2, 1, 2, 1]
profits = [100, 19, 27, 25]

job_data = []

for i in range(len(jobs)):
    job_data.append((jobs[i], deadlines[i], profits[i]))

# Sort jobs by profit in descending order
job_data.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

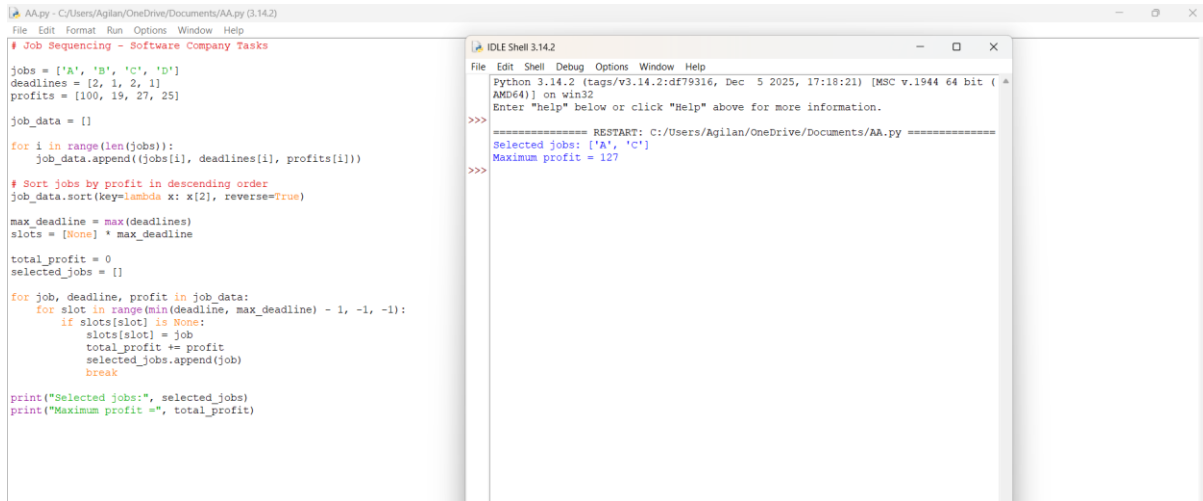
total_profit = 0
selected_jobs = []

for job, deadline, profit in job_data:
    for slot in range(min(deadline, max_deadline) - 1, -1, -1):
        if slots[slot] is None:
            slots[slot] = job
            total_profit += profit
            selected_jobs.append(job)
            break

print("Selected jobs:", selected_jobs)
print("Maximum profit =", total_profit)

```

EXECUTION & OUTPUT:



JOB SEQUENCING – EXAM PAPER EVALUATION

PYTHON CODE:

Job Sequencing - Exam Paper Evaluation

jobs = ['J1', 'J2', 'J3', 'J4', 'J5']

deadlines = [2, 1, 2, 1, 3]

profits = [20, 15, 10, 5, 1]

job_data = []

for i in range(len(jobs)):

 job_data.append((jobs[i], deadlines[i], profits[i]))

job_data.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(deadlines)

slots = [None] * max_deadline

total_profit = 0

selected_jobs = []

```

for job, deadline, profit in job_data:

    for slot in range(min(deadline, max_deadline) - 1, -1, -1):

        if slots[slot] is None:

            slots[slot] = job

            total_profit += profit

            selected_jobs.append(job)

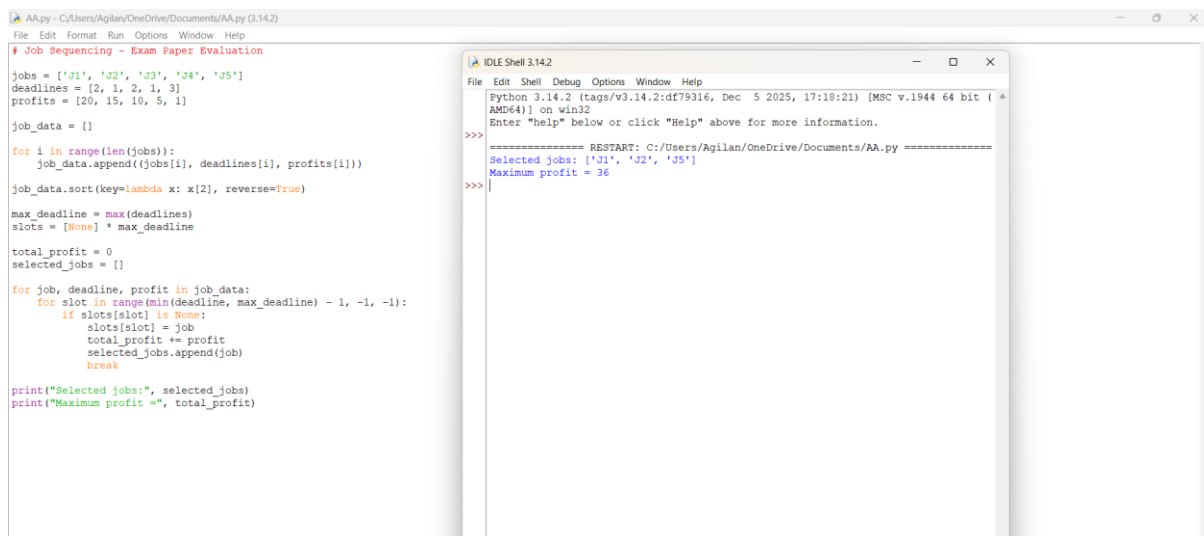
            break

print("Selected jobs:", selected_jobs)

print("Maximum profit =", total_profit)

```

EXECUTION & OUTPUT:



The screenshot shows a Python IDE window with a file named 'AA.py'. The code in the file is as follows:

```

# Job Sequencing - Exam Paper Evaluation

jobs = ['J1', 'J2', 'J3', 'J4', 'J5']
deadlines = [2, 1, 2, 1, 3]
profits = [20, 15, 10, 5, 1]

job_data = []

for i in range(len(jobs)):
    job_data.append((jobs[i], deadlines[i], profits[i]))

job_data.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0
selected_jobs = []

for job, deadline, profit in job_data:
    for slot in range(min(deadline, max_deadline) - 1, -1, -1):
        if slots[slot] is None:
            slots[slot] = job
            total_profit += profit
            selected_jobs.append(job)
            break

print("Selected jobs:", selected_jobs)
print("Maximum profit =", total_profit)

```

The output window shows the following results:

```

===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Selected jobs: ['J1', 'J2', 'J5']
Maximum profit = 36

```

JOB SEQUENCING – PRINTING PRESS ORDER

PYTHON CODE:

```
# Job Sequencing - Printing Press Orders
```

```
jobs = ['A', 'B', 'C', 'D', 'E']
```

```
deadlines = [2, 1, 2, 1, 3]
```

```
profits = [100, 50, 10, 20, 30]
```

```
job_data = []
```

```
for i in range(len(jobs)):
```

```
    job_data.append((jobs[i], deadlines[i], profits[i]))
```

```
job_data.sort(key=lambda x: x[2], reverse=True)
```

```
max_deadline = max(deadlines)
```

```
slots = [None] * max_deadline
```

```
total_profit = 0
```

```
selected_jobs = []
```

```
for job, deadline, profit in job_data:
```

```
    for slot in range(min(deadline, max_deadline) - 1, -1, -1):
```

```
        if slots[slot] is None:
```

```
            slots[slot] = job
```

```
            total_profit += profit
```

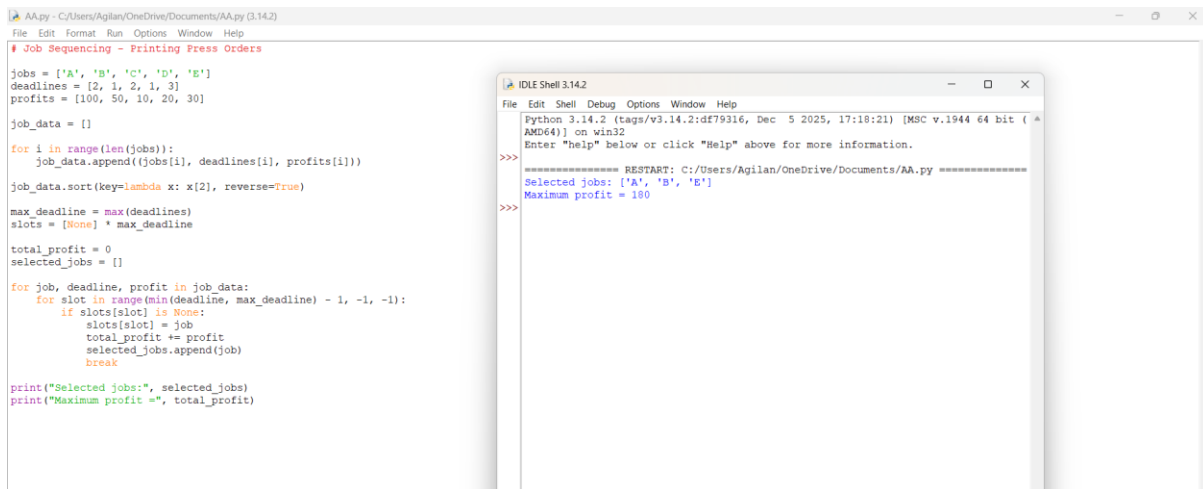
```
            selected_jobs.append(job)
```

```
            break
```

```
print("Selected jobs:", selected_jobs)
```

```
print("Maximum profit =", total_profit)
```

EXECUTION & OUTPUT:



JOB SEQUENCING – FREELANCE PROJECTS

PYTHON CODE:

Job Sequencing - Freelance Projects

jobs = ['P1', 'P2', 'P3', 'P4']

deadlines = [1, 1, 2, 2]

profits = [40, 30, 20, 10]

job_data = []

for i in range(len(jobs)):

 job_data.append((jobs[i], deadlines[i], profits[i]))

job_data.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(deadlines)

slots = [None] * max_deadline

total_profit = 0

selected_jobs = []

```

for job, deadline, profit in job_data:

    for slot in range(min(deadline, max_deadline) - 1, -1, -1):

        if slots[slot] is None:

            slots[slot] = job

            total_profit += profit

            selected_jobs.append(job)

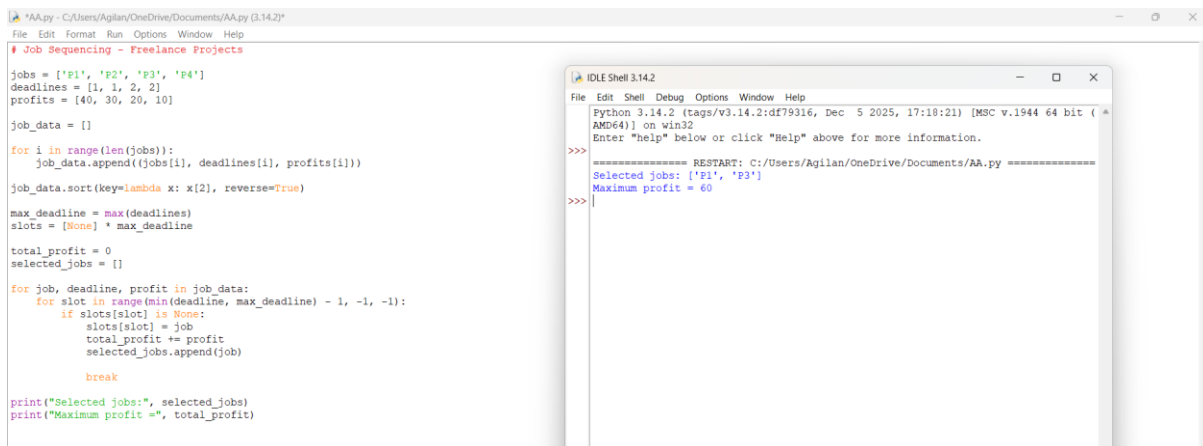
            break

print("Selected jobs:", selected_jobs)

print("Maximum profit =", total_profit)

```

EXECUTION & OUTPUT:



The screenshot shows a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The code in the editor is as follows:

```

# Job Sequencing - Freelance Projects

jobs = ['P1', 'P2', 'P3', 'P4']
deadlines = [1, 1, 2, 2]
profits = [40, 30, 20, 10]

job_data = []

for i in range(len(jobs)):
    job_data.append((jobs[i], deadlines[i], profits[i]))

job_data.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0
selected_jobs = []

for job, deadline, profit in job_data:
    for slot in range(min(deadline, max_deadline) - 1, -1, -1):
        if slots[slot] is None:
            slots[slot] = job
            total_profit += profit
            selected_jobs.append(job)
            break

print("Selected jobs:", selected_jobs)
print("Maximum profit =", total_profit)

```

The output window shows the following results:

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Selected jobs: ['P1', 'P3']
Maximum profit = 60

```

HUFFMAN CODING – TEXT COMPRESSION

PYTHON CODE:

Huffman Coding - Text Compression

```
import heapq
```

```
characters = ['a', 'b', 'c', 'd', 'e', 'f']
```

```
frequencies = [5, 9, 12, 13, 16, 45]
```

```
heap = []
```

```
for char, freq in zip(characters, frequencies):
```

```
    heapq.heappush(heap, (freq, char, char))
```

```
while len(heap) > 1:
```

```
    freq1, _, left = heapq.heappop(heap)
```

```
    freq2, _, right = heapq.heappop(heap)
```

```
    new_node = left + right
```

```
    heapq.heappush(heap, (freq1 + freq2, new_node, (left, right)))
```

```
def generate_codes(node, code="", codes=None):
```

```
    if codes is None:
```

```
        codes = {}
```

```
    if isinstance(node, str):
```

```
        codes[node] = code
```

```
    return codes
```

```
left, right = node
```

```
generate_codes(left, code + "0", codes)
```

```
generate_codes(right, code + "1", codes)
```

```
return codes
```

```

root = heap[0][2]

codes = generate_codes(root)

print("Huffman Codes:")

for char in characters:

    print(char + ":", codes[char])

```

EXECUTION & OUTPUT:

The screenshot shows a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The code is for Huffman Coding - Text Compression. It imports the 'heapq' module, defines characters and frequencies, and implements a Huffman tree building algorithm. The code then prints the Huffman codes for each character. A second window titled 'IDLE Shell 3.14.2' shows the execution output, which includes a 'RESTART' message and a 'TypeError: can only concatenate tuple (not "str") to tuple' error. The error message indicates that the code attempted to concatenate a tuple with a string, which is not allowed in Python.

```

# Huffman Coding - Text Compression

import heapq

characters = ['a', 'b', 'c', 'd', 'e', 'f']
frequencies = [5, 9, 12, 13, 16, 45]

heap = []

for char, freq in zip(characters, frequencies):
    heapq.heappush(heap, (freq, char, char))

while len(heap) > 1:
    freq1, _, left = heapq.heappop(heap)
    freq2, _, right = heapq.heappop(heap)

    new_node = left + right
    heapq.heappush(heap, (freq1 + freq2, new_node, (left, right)))

def generate_codes(node, code="", codes=None):
    if codes is None:
        codes = {}

    if isinstance(node, str):
        codes[node] = code
        return codes

    left, right = node

    generate_codes(left, code + "0", codes)
    generate_codes(right, code + "1", codes)

    return codes

root = heap[0][2]
codes = generate_codes(root)

print("Huffman Codes:")

for char in characters:
    print(char + ":", codes[char])

```

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Traceback (most recent call last):
  File "C:/Users/Agilan/OneDrive/Documents/AA.py", line 17, in <module>
    new_node = left + right
TypeError: can only concatenate tuple (not "str") to tuple

>>>

```

HUFFMAN CODING – MESSAGE ENCODING

PYTHON CODE:

Huffman Coding - Message Encoding

```

import heapq

characters = ['A', 'B', 'C', 'D']

frequencies = [10, 20, 30, 40]

```



```
heap = []
```

```
for char, freq in zip(characters, frequencies):
```

```
    heapq.heappush(heap, (freq, char, char))
```

```
while len(heap) > 1:
```

```
    freq1, _, left = heapq.heappop(heap)
```

```
    freq2, _, right = heapq.heappop(heap)
```

```
    new_node = left + right
```

```
    heapq.heappush(
```

```
        heap,
```

```
        (freq1 + freq2, new_node, (left, right))
```

```
    )
```

```
def generate_codes(node, code="", codes=None):
```

```
    if codes is None:
```

```
        codes = {}
```

```
    if isinstance(node, str):
```

```
        codes[node] = code
```

```
    return codes
```

```
left, right = node
```

```
generate_codes(left, code + "0", codes)
```

```
generate_codes(right, code + "1", codes)
```

```
return codes
```

```
root = heap[0][2]
```

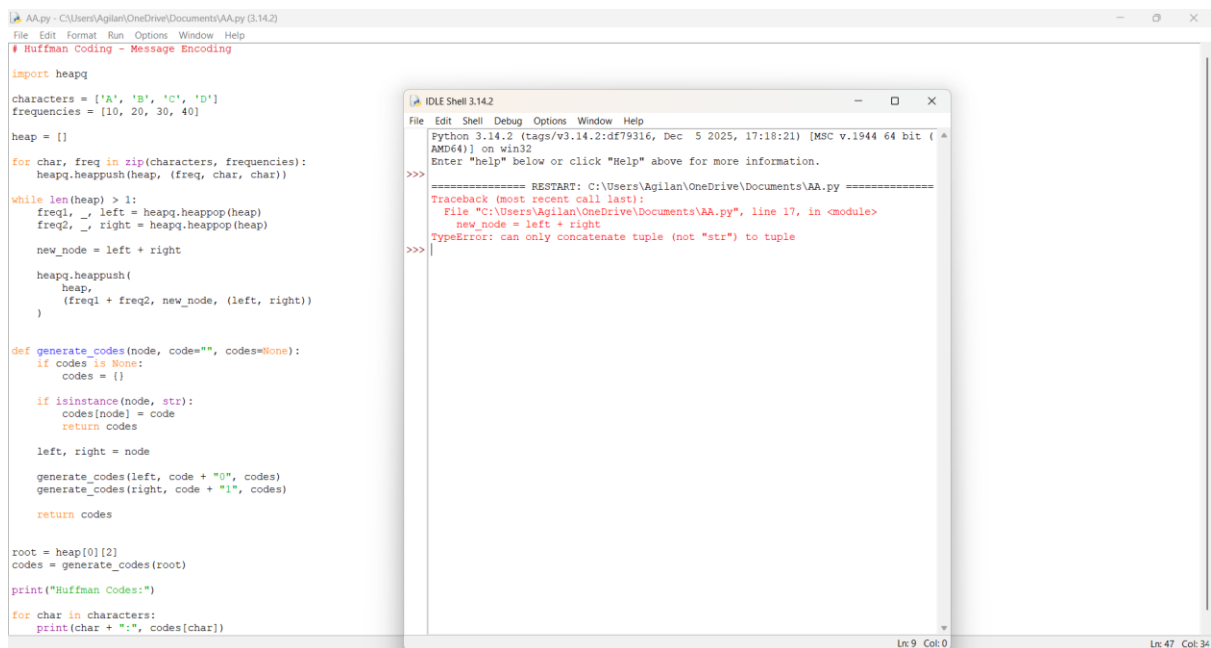
```
codes = generate_codes(root)
```

```
print("Huffman Codes:")
```

```
for char in characters:
```

```
    print(char + ":", codes[char])
```

EXECUTION & OUTPUT:



The screenshot shows two windows from a Python IDE. The main window displays a Python script for Huffman coding. The script defines characters, frequencies, and a heap to build a Huffman tree. It also includes a function to generate codes for each node. The script concludes by printing the Huffman codes for each character. A second window, titled 'IDLE Shell 3.14.2', shows the execution output. It displays a 'Restart' message and a 'Traceback' error. The error message states: 'File "C:\Users\Agilan\OneDrive\Documents\AA.py", line 17, in <module> new_node = left + right' followed by 'TypeError: can only concatenate tuple (not "str") to tuple'. This indicates a type mismatch in the code where a string and a tuple are being concatenated.

```
AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)
File Edit Format Run Options Window Help
# Huffman Coding - Message Encoding

import heapq

characters = ['A', 'B', 'C', 'D']
frequencies = [10, 20, 30, 40]

heap = []

for char, freq in zip(characters, frequencies):
    heapq.heappush(heap, (freq, char, char))

while len(heap) > 1:
    freq1, _, left = heapq.heappop(heap)
    freq2, _, right = heapq.heappop(heap)
    new_node = left + right
    heapq.heappush(heap, (freq1 + freq2, new_node, (left, right)))

def generate_codes(node, code="", codes=None):
    if codes is None:
        codes = {}
    if isinstance(node, str):
        codes[node] = code
        return codes
    left, right = node
    generate_codes(left, code + "0", codes)
    generate_codes(right, code + "1", codes)
    return codes

root = heap[0][2]
codes = generate_codes(root)

print("Huffman Codes:")

for char in characters:
    print(char + ":", codes[char])

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:dff79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Agilan\OneDrive\Documents\AA.py =====
Traceback (most recent call last):
  File "C:\Users\Agilan\OneDrive\Documents\AA.py", line 17, in <module>
    new_node = left + right
TypeError: can only concatenate tuple (not "str") to tuple
>>>
```

HUFFMAN CODING – SENSOR DATA COMPRESSION

PYTHON CODE:

```
# Huffman Coding - Sensor Data Compression
```

```
import heapq
```

```
characters = ['x', 'y', 'z', 'w']
```

```
frequencies = [7, 8, 14, 25]
```

```
heap = []
```

```
for char, freq in zip(characters, frequencies):
```

```
    heapq.heappush(heap, (freq, char, char))
```

```
while len(heap) > 1:
```

```
    freq1, _, left = heapq.heappop(heap)
```

```
    freq2, _, right = heapq.heappop(heap)
```

```
    new_node = left + right
```

```
    heapq.heappush(
```

```
        heap,
```

```
        (freq1 + freq2, new_node, (left, right))
```

```
    )
```

```
def generate_codes(node, code="", codes=None):
```

```
    if codes is None:
```

```
        codes = {}
```

```
    if isinstance(node, str):
```

```
        codes[node] = code
```

```
    return codes
```

```
left, right = node
```

```
generate_codes(left, code + "0", codes)
```

```
generate_codes(right, code + "1", codes)
```

```
return codes
```

```
root = heap[0][2]
```

```
codes = generate_codes(root)
```

```
print("Huffman Codes:")
```

```
for char in characters:
```

```
    print(char + ":", codes[char])
```

EXECUTION & OUTPUT:

```

AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)
File Edit Format Run Options Window Help
# Huffman Coding - Sensor Data Compression

import heapq

characters = ['x', 'y', 'z', 'w']
frequencies = [7, 8, 14, 25]

heap = []

for char, freq in zip(characters, frequencies):
    heapq.heappush(heap, (freq, char, char))

while len(heap) > 1:
    freq1, _, left = heapq.heappop(heap)
    freq2, _, right = heapq.heappop(heap)

    new_node = left + right

    heapq.heappush(
        heap,
        (freq1 + freq2, new_node, (left, right))
    )

def generate_codes(node, code="", codes=None):
    if codes is None:
        codes = {}

    if isinstance(node, str):
        codes[node] = code
        return codes

    left, right = node

    generate_codes(left, code + "0", codes)
    generate_codes(right, code + "1", codes)

    return codes

root = heap[0][2]
codes = generate_codes(root)

print("Huffman Codes:")

for char in characters:
    print(char + ":", codes[char])

```

COIN CHANGE – VENDING MACHINE

PYTHON CODE:

Coin Change - Vending Machine

coins = [1, 2, 5]

amount = 11

coins.sort(reverse=True)

```

result = []

remaining = amount

for coin in coins:

    while remaining >= coin:

        result.append(coin)

        remaining -= coin

print("Coins used:", result)

print("Minimum coins =", len(result))

```

EXECUTION & OUTPUT:

```

# Coin Change - Vending Machine

coins = [1, 2, 5]
amount = 11

coins.sort(reverse=True)

result = []
remaining = amount

for coin in coins:
    while remaining >= coin:
        result.append(coin)
        remaining -= coin

print("Coins used:", result)
print("Minimum coins =", len(result))

```

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1914 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\Agilan\OneDrive\Documents\AA.py =====
Coins used: [5, 5, 1]
Minimum coins = 3
>>>

```

FRACTIONAL KNAPSACK – WAREHOUSE PACKING

PYTHON CODE:

```
# Fractional Knapsack - Warehouse Packing
```

```
weights = [2, 3, 5, 7]
```

```
profits = [10, 5, 15, 7]
```

```
capacity = 8
```

```
items = []
```

```
for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

# Sort by profit/weight ratio
items.sort(reverse=True)

total_profit = 0
remaining_capacity = capacity

for ratio, weight, profit in items:

    if remaining_capacity >= weight:
        total_profit += profit
        remaining_capacity -= weight

    else:
        fraction = remaining_capacity / weight
        total_profit += profit * fraction
        break

print("Maximum profit =", total_profit)
```

EXECUTION & OUTPUT:

```
AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)
File Edit Format Run Options Window Help

# Fractional Knapsack - Warehouse Packing

weights = [2, 3, 5, 7]
profits = [10, 5, 15, 7]
capacity = 8

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

# Sort by profit/weight ratio
items.sort(reverse=True)

total_profit = 0
remaining_capacity = capacity

for ratio, weight, profit in items:
    if remaining_capacity >= weight:
        total_profit += profit
        remaining_capacity -= weight
    else:
        fraction = remaining_capacity / weight
        total_profit += profit * fraction
        break

print("Maximum profit =", total_profit)
```

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\Agilan\OneDrive\Documents\AA.py =====
>>> Maximum profit = 26.666666666666668
>>>
```